VIKUNA TECHNOLOGIES

Proprietary LLM Strategy

Interaction Logging, Fine-Tuning & IP Moat

DristiQ · KI-Prime · ContractNest

Version 1.0 | May 2026 | Internal Strategy Document

Confidential — Vikuna Technologies

1. Executive Summary

Vikuna Technologies operates three SaaS products — DristiQ, KI-Prime, and ContractNest — each generating user interactions with an LLM inference layer. This document defines the strategy to systematically capture those interactions, convert them into proprietary training data, and progressively fine-tune a domain-specific LLM that becomes a defensible competitive asset.

The core insight: every useful LLM response delivered to a user is a potential training example. Products that log these interactions from day one compound their AI quality advantage over time. Products that do not, remain permanently dependent on generic models.

Strategic Objective

Convert daily product interactions across DristiQ, KI-Prime, and ContractNest into a proprietary fine-tuned LLM (Vikuna-LLM-1.0) within 12 months — creating an IP moat that cannot be replicated without replicating our operational journey.

1.1 Three Approaches — Clarity

Approach	What It Does	Cost	Our Path
Full Training	Build LLM from scratch	Millions USD + GPU cluster	Not applicable
Fine-Tuning	Adapt existing model on domain data	~$20–50 per training run	Target — Phase 3
RAG	Retrieve context at inference time	Near zero	Now — Phase 1
Interaction Log	Capture interactions for future fine-tuning	Minimal	Now — Phase 1

1.2 Quality Gap — Generic vs Fine-Tuned

Query	Generic Qwen3	Fine-Tuned Vikuna-LLM
Tithi is Krishna Trayodashi, BANKNIFTY bearish engulfing — what does this mean?	Generic moon phase + candlestick explanation	sessionQuality=1, Honest Score drops below 40, SVD aligns, avoid fresh longs before Rahu Kala 14:00
Client XIRR dropped from 14% to 9% — what should I tell them?	Generic advisor language about market cycles	Structured KI-Prime client communication with portfolio-specific context and next action
Generate AMC service contract for HVAC unit — 2 visits/year	Generic contract template	ContractNest-aware contract with equipment checkpoints, SLA terms, and compliance clauses

2. IP & Competitive Moat

2.1 How Fine-Tuning Creates IP

Fine-tuning does not mean owning the base model architecture. It means the domain reasoning, terminology, output patterns, and business logic of Vikuna products get encoded into model weights — weights that are private, not publicly downloadable, and cannot be reverse-engineered.

Asset	Before Fine-Tuning	After Fine-Tuning
Honest Score formula (60/20/20)	In codebase — visible if leaked	Encoded in weights — opaque
System prompts	In API calls — interceptable	Implicit in model behaviour
Domain terminology	Injected every call	Native to model
Output format	Enforced via prompt	Natural model output
Competitive replicability	High — prompts can be copied	Low — data cannot be copied

2.2 The Data Flywheel

The compounding mechanism that creates a durable moat:

More Users → More Interactions Logged → Better Training Data
↓
Fine-Tuning Run → Better Model → Better Product
↑

Every competitor starting from scratch has zero of this interaction data. Vikuna's 6–12 month head start compounds with each product launch.

2.3 Licensing Potential

Opportunity	Description	Timeline
DristiQ-LLM API	License Vedic astrology × trading LLM to other platforms	12–18 months
KI-Prime-LLM API	MFD-specific LLM for Indian wealth management platforms	12–18 months
ContractNest-LLM API	Contract generation LLM for field service platforms	18–24 months
Vikuna-LLM-1.0	Unified model across all three domains — enterprise license	24 months

Base Model Licence Note

Qwen3 is Apache 2.0 licensed — commercial use of fine-tuned versions is permitted. The fine-tuned weights, training dataset, and domain reasoning encoded therein are Vikuna Technologies IP.

3. Interaction Logging Schema

A single shared table with product-specific context payloads. Implemented once in VaNi framework — consumed by all three products.

3.1 Core Table — vn_interaction_log

```
-- Shared across DristiQ, KI-Prime, ContractNest
-- DB: vani_db (VPS: 187.127.136.65)

CREATE TABLE vn_interaction_log (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product           TEXT NOT NULL,        -- 'dristiq' | 'ki_prime' | 'contractnest'
  tenant_id         UUID,                 -- multi-tenant support
  session_id        UUID,                 -- groups a conversation turn
  user_id           UUID,

  -- LLM Call
  system_prompt     TEXT,
  user_input        TEXT NOT NULL,
  context_payload   JSONB,                -- product-specific structured context
  llm_response      TEXT NOT NULL,
  model_version     TEXT,                 -- 'qwen3-4b-q4km' | 'vikuna-llm-1.0'

  -- Quality Signals
  user_rating       SMALLINT,             -- 1-5 explicit rating
  was_edited        BOOLEAN DEFAULT FALSE,
  edited_response   TEXT,                 -- gold standard if user corrected
  follow_up_query   TEXT,                 -- signals confusion / dissatisfaction
  was_accepted      BOOLEAN,              -- user acted on the response

  -- Performance
  prompt_tokens     INT,
  completion_tokens INT,
  latency_ms        INT,
  endpoint          TEXT,                 -- 'llm.dristiq.com'

  created_at        TIMESTAMPTZ DEFAULT now()
);

-- Indexes for export pipeline
CREATE INDEX idx_vil_product   ON vn_interaction_log(product);
CREATE INDEX idx_vil_quality   ON vn_interaction_log(user_rating, was_edited);
CREATE INDEX idx_vil_created   ON vn_interaction_log(created_at);
```

3.2 Quality Signal Hierarchy

Signal	Weight	Meaning	How Captured
edited_response populated	Highest	User wrote the correct answer — pure gold	UI edit action
user_rating = 5	High	Explicit approval	Thumbs up / star
was_accepted = true	High	User acted on recommendation	Action event
No follow_up within 60s	Medium	Response was sufficient	Session timer
user_rating = 3-4	Medium	Acceptable but not perfect	Rating widget
follow_up_query populated	Negative	User was confused — response inadequate	Next query logged
user_rating = 1-2	Exclude	Bad response — exclude from training	Rating widget

3.3 Product Context Payloads

DristiQ Context

```
{
  "date": "2026-05-15",
  "tithi": "Krishna Trayodashi",
  "nakshatra": "Anuradha",
  "yoga": "Shiva",
  "session_quality": 1,
  "rahu_kala": "14:00-15:30",
  "abhijit_muhurta": "11:48-12:36",
  "index": "BANKNIFTY",
  "ltp": 48450,
  "signal_type": "SVD",
  "honest_score": 38,
  "conviction_tier": "indicative",
  "planetary_score": 0.45,
  "tech_score": 0.52
}
```

KI-Prime Context

```
{
  "skill": "portfolio",
  "client_id": "uuid",
  "portfolio_value": 2850000,
  "xirr": 14.2,
  "top_fund": "Parag Parikh Flexi Cap",
  "risk_profile": "moderate",
  "goal_type": "wealth_creation",
  "horizon_years": 7,
  "drift_alert": true,
  "review_due": true
}
```

ContractNest Context

```
{
  "skill": "contract_generation",
  "tenant_id": "uuid",
  "equipment_type": "HVAC",
  "equipment_variant": "Split AC 1.5T",
  "service_visits": 2,
  "contract_type": "AMC",
  "industry": "healthcare",
  "sla_response_hours": 4,
  "compliance_required": ["NABH"]
}
```

4. Implementation — Per Product

Each product implements a logging middleware that intercepts every LLM call, logs the interaction, and returns the response transparently. Claude Code executes in each respective repo.

4.1 DristiQ — FastAPI Middleware

Repo: kaala-dristi | Backend: kd-pipeline-api2 (FastAPI, port 8101)

```
# File: app/middleware/interaction_logger.py
```

```
import uuid, time
from app.db import get_db
from app.llm import call_qwen

async def log_llm_interaction(
    user_input: str,
    context_payload: dict,
    system_prompt: str,
    session_id: uuid.UUID,
    user_id: uuid.UUID = None
) -> str:
    start = time.monotonic()
    response = await call_qwen(system_prompt, user_input, context_payload)
    latency = int((time.monotonic() - start) * 1000)

    await get_db().execute("""
        INSERT INTO vn_interaction_log (
            product, session_id, user_id,
            system_prompt, user_input, context_payload,
            llm_response, model_version,
            prompt_tokens, completion_tokens, latency_ms, endpoint
        ) VALUES (
            'dristiq', $1, $2, $3, $4, $5, $6,
            'qwen3-4b-q4km', $7, $8, $9, 'llm.dristiq.com'
        )""",
        session_id, user_id, system_prompt, user_input,
        json.dumps(context_payload), response,
        response.usage.prompt_tokens,
        response.usage.completion_tokens,
        latency
    )
    return response.content
```

4.2 KI-Prime — Express Middleware

Repo: KI-Prime | Backend: Express.js

```
// File: src/middleware/interactionLogger.js

const { pool } = require('../db');

async function logLLMInteraction({
  skill, userId, tenantId, sessionId,
  systemPrompt, userInput, contextPayload,
  llmResponse, usage, latencyMs
}) {
  await pool.query(`
    INSERT INTO vn_interaction_log (
      product, tenant_id, session_id, user_id,
      system_prompt, user_input, context_payload,
      llm_response, model_version,
      prompt_tokens, completion_tokens, latency_ms, endpoint
    ) VALUES (
      'ki_prime', $1, $2, $3, $4, $5, $6,
      $7, 'qwen3-4b-q4km', $8, $9, $10, 'llm.dristiq.com'
    )`,
    [tenantId, sessionId, userId,
     systemPrompt, userInput, JSON.stringify(contextPayload),
     llmResponse, usage.prompt_tokens,
     usage.completion_tokens, latencyMs]
  );
}

module.exports = { logLLMInteraction };
```

4.3 ContractNest — Express Middleware

Repo: contractnest-combined | Backend: Express.js API

```
// File: api/src/middleware/interactionLogger.js
// Same pattern as KI-Prime — product = 'contractnest'
// context_payload includes: equipment_type, contract_type,
//   industry, tenant_id, sla_response_hours, compliance_required
```

4.4 Quality Signal Collection — UI Components

Each product adds a minimal feedback widget after LLM responses:

Signal	UI Element	DB Update
Thumbs up	Icon button	user_rating = 5
Thumbs down	Icon button	user_rating = 1
Edit response	Edit icon on response	was_edited = true, edited_response = <text>
Accept/Act	CTA button click	was_accepted = true
No follow-up	Passive — 60s timer	Inferred quality signal in export pipeline

4.5 Claude Code Task Specs

DristiQ — Claude Code Task

In kaala-dristi repo: (1) Create app/middleware/interaction_logger.py with log_llm_interaction() function. (2) Wire into all existing LLM call sites in kd-pipeline-api2. (3) Create migration M071 to add vn_interaction_log table to kaala_dristi_db. (4) Add thumbs up/down component to signal explanation cards in frontend.

KI-Prime — Claude Code Task

In KI-Prime repo: (1) Create src/middleware/interactionLogger.js. (2) Wire into all skill response handlers. (3) Create migration to add vn_interaction_log to ki_prime_db. (4) Add feedback widget to VaNi response components in UI.

ContractNest — Claude Code Task

In contractnest-combined/api: (1) Create interactionLogger.js middleware. (2) Wire into Smart Forms LLM calls and Global Template Designer AI calls. (3) Create migration for contractnest DB. (4) Add feedback widget to AI-generated contract previews.

5. Fine-Tuning Pipeline

5.1 Data Export — Monthly

```
-- Export high-quality interactions as fine-tuning dataset
-- Run monthly once sufficient volume accumulated
```

```
SELECT
  system_prompt,
  user_input,
  context_payload,
  COALESCE(edited_response, llm_response) AS ideal_response,
  product,
  created_at
FROM vn_interaction_log
WHERE
  (user_rating >= 4 OR was_accepted = true OR
   (was_edited = false AND follow_up_query IS NULL))
  AND user_rating != 1
  AND user_rating != 2
  AND llm_response IS NOT NULL
ORDER BY
  CASE WHEN edited_response IS NOT NULL THEN 0  -- highest priority
       WHEN user_rating = 5 THEN 1
       WHEN was_accepted = true THEN 2
       ELSE 3
  END,
  created_at DESC;
```

5.2 JSONL Format — Fine-Tuning Dataset

```
# Each row from export becomes one JSONL line
{
  "messages": [
    {
      "role": "system",
      "content": "<system_prompt>"
    },
    {
      "role": "user",
      "content": "Context: <context_payload>\n\nQuery: <user_input>"
    },
    {
      "role": "assistant",
      "content": "<ideal_response>"
    }
  ]
}
```

5.3 Fine-Tuning Milestones

Milestone	Dataset Size	Expected Quality	Training Cost
First run	500+ examples	Noticeable domain vocabulary improvement	~$20 (RunPod A100)
Strong run	2,000+ examples	Reliable domain reasoning, correct output format	~$50
Vikuna-LLM-1.0	5,000+ examples	Full domain specialisation across all 3 products	~$100

5.4 Fine-Tuning Method — QLoRA

QLoRA (Quantized Low-Rank Adaptation) is the practical approach — fine-tunes only a small adapter layer on top of frozen base model weights. Runs on a single A100 GPU rented by the hour. Produces LoRA adapter files that merge into base Qwen3 weights for deployment.

```
# Fine-tuning run (RunPod A100 — ~$1.50/hr)
```

```
# Expected duration: 2-4 hours per training run

pip install transformers peft trl datasets

python fine_tune.py \
  --model_name Qwen/Qwen3-4B \
  --dataset vikuna_interactions.jsonl \
  --output_dir ./vikuna-llm-v1 \
  --num_train_epochs 3 \
  --per_device_train_batch_size 4 \
  --lora_r 16 \
  --lora_alpha 32 \
  --quantization 4bit
```

5.5 Deployment — Swap In Place

After fine-tuning, merge LoRA adapters into base weights and convert to GGUF. Drop into /opt/vikuna/models/ on LLM VPS. Restart vikuna-llm container. Zero downtime swap.

```
# Merge + convert to GGUF
python merge_lora.py --base Qwen/Qwen3-4B --adapter ./vikuna-llm-v1
python convert_to_gguf.py --model ./vikuna-llm-merged --quantize Q4_K_M

# Deploy on LLM VPS
scp vikuna-llm-v1-Q4_K_M.gguf root@72.60.222.136:/opt/vikuna/models/
ssh root@72.60.222.136
  # Update docker-compose model filename
  docker compose up -d vikuna-llm
```

6. Phased Roadmap

Phase 1 Now → Month 3 — **Foundation — Log Everything**

Task	Product	Owner
Implement vn_interaction_log table in each product DB	All three	Claude Code
FastAPI logging middleware in kd-pipeline-api2	DristiQ	Claude Code
Express logging middleware in KI-Prime	KI-Prime	Claude Code
Express logging middleware in ContractNest API	ContractNest	Claude Code
Thumbs up/down UI component in each product	All three	Claude Code
Edit response capture in DristiQ signal explanations	DristiQ	Claude Code
Wire llm.dristiq.com endpoint into all products	All three	Claude Code

Phase 2 Month 3 → Month 9 — **Quality & Volume — Build the Dataset**

Task	Target
500+ DristiQ interactions logged	Signal explanations, muhurta queries
500+ KI-Prime interactions logged	Portfolio reviews, client communications
300+ ContractNest interactions logged	Contract generation, Smart Forms AI
Monthly export pipeline running	Auto-export to JSONL dataset
Quality review — manual spot-check of edited responses	Ensure gold standard accuracy
First fine-tuning test run	Validate pipeline end-to-end

Phase 3
Month 9 → Month 12

Vikuna-LLM-1.0 — Proprietary Model

Task	Target
2,000+ curated interactions across all three products	Cross-domain fine-tuning dataset
QLoRA fine-tuning run on RunPod A100	~$50, 4 hours
Vikuna-LLM-1.0 deployed on llm.dristiq.com	Drop-in replacement for Qwen3
A/B test — Qwen3 vs Vikuna-LLM-1.0 on real queries	Measure quality delta
Per-client API key billing implemented	Monetise external access
Licensing conversations with target platforms	DristiQ-LLM for astro-finance apps

7. Data Volume Projections

Product	Interactions/Day (est.)	Month 3	Month 6	Month 12
DristiQ	10–30	900	1,800	3,600+
KI-Prime	20–50	1,800	3,600	7,200+
ContractNest	5–15	450	900	1,800+
TOTAL	35–95	3,150	6,300	12,600+

Assumes beta user base of 10–30 active users per product. First fine-tuning milestone (500 high-quality examples) reached by Month 2–3. Production fine-tune (2,000+ examples) achievable by Month 6.

Quality over Quantity

500 gold-standard examples (user-edited responses, 5-star ratings, accepted actions) outperform 5,000 unfiltered logs. The quality signal capture in the UI is as important as the volume of interactions.

8. Cost Summary

Item	Cost	When	Notes
Interaction logging implementation	0	Now	Claude Code — 1–2 days per product
Storage (vn_interaction_log)	~0	Ongoing	Text data — negligible on existing VPS
Monthly export pipeline	0	Month 3+	SQL query + Python script
First fine-tuning test run	~$20	Month 3	RunPod A100 — 2 hours
Vikuna-LLM-1.0 training run	~$50	Month 9	RunPod A100 — 4 hours
Subsequent training runs	~$50 each	Quarterly	As dataset grows
LLM VPS (already running)	₹799/mo	Ongoing	Qwen3 → Vikuna-LLM-1.0 same infra
TOTAL to Vikuna-LLM-1.0	~$70	Month 9	Excluding VPS already budgeted

ROI Note

At ~$70 total investment to reach Vikuna-LLM-1.0, this is among the highest ROI AI initiatives available to a product company at Vikuna's stage. The resulting model is a licensable asset and competitive differentiator that would cost competitors 12+ months and significant capital to replicate.

End of Document

Confidential | Vikuna Technologies | May 2026